Qwen

Building a High-Performance Game Bot: A Python-Based Architecture Combining DirectX Capture, PaddleOCR, and a Secure Lua-like Scripting Engine

Core System Architecture and Real-Time Performance Optimization

The development of a general-purpose PC game bot necessitates an architecture centered on real-time performance, robustness, and efficiency. The foundational component of this architecture is the screen capture engine, as its throughput and latency directly dictate the bot's ability to react to in-game events. An analysis of available technologies reveals a clear distinction between high-performance, Windows-specific solutions and more generalized, but slower, cross-platform alternatives. For any application demanding real-time responsiveness, particularly in competitive gaming contexts, prioritizing the highest possible frame rate is non-negotiable. The choice of a screen capture library is therefore a critical architectural decision that will influence every subsequent layer of the system.

The most effective approach for capturing game screens, especially for modern, DirectX-based applications running in exclusive fullscreen mode, is to leverage the Windows Desktop Duplication API [126]. This native API provides low-level access to desktop frames, making it ideal for applications requiring low-latency video capture, such as game streaming services [167]. Several Python libraries have been developed to harness this powerful API, with `DXcam` emerging as a prominent and highly optimized solution. `DXcam` is explicitly designed for high-throughput, low-latency screen capture on Windows, capable of achieving frame rates exceeding 240 FPS even at 1080p resolution [28] [31]. Its stability with fullscreen exclusive Direct3D applications is a key advantage over other methods that often fail or stutter in this mode [31]. Other libraries built on the same principle include `D3DShot`, another pure Python implementation of the Desktop Duplication API, and `BetterCam`, which is also noted as one of the world's fastest publicly available Python screenshot libraries for Windows [164] [170] [260]. These libraries provide the raw performance required to feed a computer vision pipeline without introducing significant processing bottlenecks. Their reliance on the Desktop Duplication

API makes them inherently Windows-only, a limitation that must be accepted if top-tier performance is the primary goal. The alternative, the Windows Graphics Capture API, offers benefits like cross-GPU support but may introduce slightly more overhead compared to DXGI/Duplication APIs [171].

In contrast, cross-platform libraries like `mss` (Multi-Screen-Shot) present a viable but significantly less performant option. While `mss` is praised for its ease of use and compatibility across Windows, Linux, and macOS, benchmarks consistently show it to be substantially slower than `DXcam` on Windows systems [29] [33]. One source indicates that `mss` can be approximately three times slower than `DXcam` when running on Windows [33]. Another analysis found that `mss` takes between 16 and 47ms per screenshot, depending on the capture area, which translates to a much lower maximum achievable FPS [86]. While `mss` might suffice for less demanding applications or for users on non-Windows operating systems, it falls short of the "real-time" performance requirement implied by the project goal, especially when combined with computationally intensive tasks like object detection and OCR. Therefore, the core architecture should be built around a Windows-first strategy utilizing a library like `DXcam` or `D3DShot`. A secondary, less performant capture path could be implemented for other platforms, but the primary development effort must focus on maximizing performance within the Windows ecosystem.

Once a high-speed capture mechanism is established, the next consideration is how to integrate it into the overall application flow. A naive implementation where the main thread captures a frame, processes it, and then waits would lead to poor performance and a sluggish user experience. The optimal design involves a producer-consumer model using threading. The screen capture module acts as the "producer," running in its own dedicated thread to continuously acquire frames from the game window as fast as the hardware and API allow. These frames (typically NumPy arrays) are then placed into a thread-safe queue. Concurrently, one or more "consumer" threads, likely managed by a thread pool, pull frames from this queue for processing. This decoupling ensures that the capture process is never blocked by a slow vision algorithm, and conversely, a slow processing task does not cause the capture buffer to overflow and drop frames. This architecture is a common pattern in real-time data processing pipelines, such as those used for combining real-time object detection (YOLOv8), tracking (SORT), and text recognition (EasyOCR) [214]. It provides the necessary resilience and parallelism to maintain high throughput. The consumer threads would handle the heavy lifting of applying various computer vision algorithms—template matching, deep learning inference, and OCR—to the captured image data. By separating concerns and leveraging multi-threading, the application can achieve the high degree of responsiveness and reliability required for a functional game bot.

Library/API	Platform Support	Max Reported FPS (1080p)	Key Advantage	Key Disadvantage
DXcam / BetterCam	Windows Only [28]	240+ FPS [31] [170]	Extremely high throughput and stability with fullscreen games via Desktop Duplication API [126].	Windows-only; requires specific setup for some games [116].
D3DShot	Windows Only [164]	Very High (specific FPS numbers not provided)	Pure Python implementation of the Desktop Duplication API [32].	Windows-only.
mss	Cross-Platform [87]	Significantly lower (approx. 3x slower than DXcam on Windows) [33]	Ease of use and broad OS compatibility [29].	Not suitable for high-demand real-time applications due to higher latency and lower throughput [33].

This architectural foundation—centered on high-performance Windows-native screen capture, integrated into a multi-threaded producer-consumer pipeline—provides the necessary performance ceiling to support the more complex computer vision and scripting components of the game bot. It ensures that the system can operate in real-time without being bottlenecked by the initial step of acquiring visual data from the game environment.

Hybrid Computer Vision Pipeline for Object and Text Recognition

A general-purpose game bot must possess versatile perception capabilities, enabling it to identify both static objects and dynamic textual information within a game's visual field. Achieving this effectively requires a hybrid computer vision pipeline that combines the strengths of different techniques. Relying on a single method, such as a deep learning model for all tasks, would be inefficient and potentially less accurate than using simpler, faster methods for appropriate problems. The proposed architecture should therefore incorporate a modular vision system with distinct pathways for object recognition via template matching and for text recognition via Optical Character Recognition (OCR). This allows the system to balance speed and accuracy by selecting the optimal tool for each specific task, a crucial factor in maintaining real-time performance.

For the specified requirement of recognizing objects from multiple image samples located in a directory like `/source/Boss/Target`, template matching is the most direct and computationally efficient approach. This technique works by sliding a small reference image (the template) over a larger source image and calculating a match score at each position using a chosen algorithm (e.g., correlation coefficient) [72] [74]. The OpenCV

library provides a robust implementation of this function (`cv2.matchTemplate`) [74]. To handle variations in scale, a multi-scale template matching approach can be employed, where the search is performed on image pyramids of different sizes [57]. This aligns perfectly with the user's request to use a folder of sample images, as the system can iterate through these templates to find any of them on the screen. The primary advantage of template matching is its speed; it is a simple pixel-wise comparison that runs very quickly, even on a CPU [59]. However, it has significant limitations: it is highly sensitive to changes in rotation, scaling, and lighting conditions [72]. Despite these weaknesses, for detecting stable UI elements, specific character models, or items with consistent visual representations, template matching remains a surprisingly powerful and accessible tool [58] [59].

For more complex object detection scenarios where targets may appear at different scales, orientations, or be partially occluded, a deep learning-based approach using models from the YOLO (You Only Look Once) family is superior. Lightweight variants like YOLOv8n, YOLOv10n, and YOLOv12n have demonstrated an excellent balance of accuracy and speed, making them well-suited for real-time inference [109][157]. Benchmarks comparing these models show that they can achieve high mean Average Precision (mAP) while maintaining extremely low latency, with some versions offering better accuracy than others at the same latency [156][174]. For instance, YOLOv10-N has shown an inference latency of only 1.84ms, significantly faster than YOLOv8-N's 6.16ms, demonstrating the rapid evolution of these models toward greater efficiency [112]. The integration of these models can be further optimized by using their ONNX (Open Neural Network Exchange) format, which allows for deployment on various hardware accelerators, including CPUs, via optimized runtimes like NVIDIA's TensorRT or Intel's OpenVINO [347][353]. Batch processing, where multiple frames are processed simultaneously, can also reduce total inference time [114]. The system's vision module should thus feature a configurable "detector" that can switch between a fast, multi-template matching mode for simple targets and a more robust, YOLO-based detector for complex scenes.

For text recognition, the choice of OCR engine is critical for balancing speed and accuracy. Among the available open-source options, PaddleOCR has consistently emerged as a strong contender. It is frequently cited as offering the best trade-off between accuracy, speed, and memory efficiency, often outperforming both Tesseract and EasyOCR [50] [52] [53]. PaddleOCR is capable of achieving high accuracy (reportedly 95%+) and is optimized for both speed and performance, making it suitable for high-volume image processing tasks [51] [54]. Crucially, PaddleOCR performs well on CPUs, especially its ONNX-optimized versions, which removes the dependency on a powerful GPU for basic

OCR needs [155][358]. This contrasts with Tesseract, which, despite being widely known, is often criticized for being slow in real-time applications [213]. While Tesseract's performance can be improved by selecting specific model data files (e.g., 'fast' models instead of 'best') or using quantized models, PaddleOCR appears to deliver better baseline performance [123] [261]. EasyOCR is another popular choice, often noted for its ease of use and good performance, but PaddleOCR generally maintains an edge in head-to-head comparisons [55] [154]. Given these findings, PaddleOCR should be the default and recommended OCR backend for the game bot. Furthermore, performance can be enhanced through image pre-processing techniques such as noise reduction, binarization, and resizing, which are known to improve both the accuracy and speed of OCR engines [263][264].

The following table compares the leading candidates for each vision task:

Task	Technique/Lib	Pros	Cons
Object Recognition	Multi-Template Matching	Very fast, computationally cheap, easy to implement, perfect for stable targets [58] [59].	Sensitive to rotation, scaling, and lighting changes [72].
Object Recognition	Lightweight YOLO Models (e.g., v10n)	Robust to occlusion and viewpoint changes, high accuracy, real-time speeds on CPU/GPU [46] [112] [157].	Higher computational cost than template matching, requires model loading.
Text Recognition (OCR)	PaddleOCR	Excellent speed/accuracy trade-off, CPU-friendly (especially ONNX version), high accuracy [50] [51] [52].	Larger dependency footprint than Tesseract.
Text Recognition (OCR)	Tesseract	Mature, wide language support, extensive community resources.	Often slow for real-time use, requires careful configuration for good results [125] [213].
Text Recognition (OCR)	EasyOCR	Good accuracy, supports many languages, easy to use [154].	Generally slower than PaddleOCR [55].

By architecting the vision system as a hybrid pipeline, the bot gains the flexibility to apply the right tool for the job. A user-defined script can specify whether to look for a target using a set of static images (template matching) or a more flexible deep learning model. Similarly, the system defaults to the high-performance PaddleOCR for any text-related commands. This modular design ensures that the bot remains responsive and accurate across a wide variety of game environments and detection challenges.

Design and Implementation of a Secure Custom Scripting Engine

A defining feature of the proposed game bot is its reliance on a custom, Lua-like scripting language for defining bot behavior. This requirement presents a significant architectural challenge that intersects with language design, security, and extensibility. The choice between embedding a full-fledged language like Python versus creating a tailored Domain-Specific Language (DSL) is a pivotal decision that impacts the entire project's complexity, safety, and user accessibility. Furthermore, regardless of the chosen path, implementing a secure sandboxing mechanism is an absolute necessity to protect the user's system from malicious or poorly written scripts.

Embedding a powerful language like Python offers immense potential. The vast ecosystem of libraries, including OpenCV for computer vision, NumPy for numerical operations, and PyAutoGUI for input simulation, would make the bot incredibly capable and easy for developers to extend [48] [303]. However, executing arbitrary user-provided Python code is a major security risk. A compromised script could gain unauthorized access to the user's file system, network, or other running processes, effectively installing malware [11] [14]. Running untrusted code is a well-understood problem in software engineering, and the standard solution is sandboxing [12] [15]. Specialized tools like `openedx/codejail` are designed specifically to manage the execution of untrusted code in secure, isolated sandboxes [18]. While powerful, integrating such a system adds significant complexity to the project. An alternative is to use a two-way bridge like `Lunatic Python`, which allows a Lua script to safely call back into a pre-approved set of functions within the main Python application, providing a controlled interface without exposing the entire Python environment [302].

Lua, on the other hand, is purpose-built as an embedded scripting language. It was designed from the ground up to be lightweight, fast, and easily integrated into C/C++ applications, which is precisely the context of this project [102] [247]. Its syntax is clean and relatively easy to learn, similar in feel to Python or Pascal, making it accessible to novice programmers [105] [249]. Many game engines, such as Defold, extensively use Lua for scripting game logic because it is safe, efficient, and easy to embed [203] [208]. The Lua interpreter is small enough that its entire source code can be included directly in a project, simplifying distribution [102]. While one contrarian opinion suggests avoiding Lua for new projects in favor of languages like Go or Rust, this overlooks Lua's specific niche and proven track record in embedded systems and game development [176] [245]. For the stated goal of

creating a "custom, Lua-like" language, adopting Lua itself would be a pragmatic and robust choice.

The third option is to build a true DSL tailored specifically for game botting. This approach offers the best of both worlds: the simplicity and safety of a custom language combined with the control and security of a host application written in Python. A DSL allows the creator to define a simple, intuitive syntax that maps directly to the bot's functionality (e.g., `if object_found("Boss/Target") then attack() end`) [103]. This avoids overwhelming the user with the complexities of a general-purpose language. Python provides excellent tools for building such languages. The `ast` module can be used to parse and analyze expressions in a controlled manner, and packages like `textX` can generate parsers and Abstract Syntax Trees (ASTs) directly from a grammar definition [179][240]. An AST is a tree representation of the source code's structure, which can then be traversed and executed by a custom interpreter written in Python [242][244]. This gives complete control over what code can be run and how it is interpreted, ensuring that potentially dangerous operations are never executed. Implementing a DSL in Python is a manageable task, even for a solo developer, and numerous tutorials and resources exist to guide the process [277][281].

Regardless of the language chosen, the security implications cannot be overstated. Allowing users to run arbitrary code is irresponsible unless a robust sandbox is in place. The table below outlines the trade-offs of each approach.

Approach	Description	Advantages	Disadvantages
Embedded Python (with Sandboxing)	Use the full Python interpreter, but run scripts in a secure, isolated environment like `CodeJail`. [18] [12]	Unmatched power and access to a vast library ecosystem.	High complexity; requires a sophisticated sandboxing solution. Security risks remain if the sandbox is flawed.
Embedded Lua	Integrate the Lua interpreter, which is designed for this purpose. [102] [247]	Lightweight, fast, easy to embed, secure by design.	Less powerful than Python; limited to the libraries available for Lua.
Custom DSL in Python	Create a simple, tailored language whose parser and interpreter are written in Python. [179] [103]	Maximum control and security; simple and intuitive for the target domain; no external interpreter needed.	Development effort required to build the language; less expressive than a full programming language.

Given the user's request for a "custom, Lua-like" language and the emphasis on a "general-purpose" tool, the most prudent path is to develop a custom DSL. This approach aligns with the desire for simplicity and safety, keeps the execution context firmly within the secure Python application, and provides a clean, extensible interface for defining bot logic. It avoids the pitfalls of sandboxing a full language while still delivering the

flexibility and power needed for complex automation tasks. The scripting engine would consist of a parser that converts the script into an AST and an evaluator that walks the tree, executing the corresponding actions (e.g., calling vision functions, sending keyboard inputs) provided by the main application.

Dynamic GUI Generation and Minimalist In-Game Overlay

The user interface (UI) of the game bot is a critical component that must serve two distinct purposes: a comprehensive configuration panel before the bot starts, and a minimal, non-obtrusive overlay during runtime. The design and implementation of these two interfaces require different technologies and philosophies. The configuration GUI must be dynamic, generating widgets based on the script's variable declarations, while the runtime overlay must prioritize extreme performance and a negligible screen footprint to avoid interfering with gameplay.

For the pre-execution configuration panel, the goal is to create a GUI that can programmatically generate controls like sliders, toggles (checkboxes), text fields, and dropdown menus based on metadata defined within the bot script. This requires the script to declare its configurable parameters with associated types and constraints (e.g., `expose_slider("attack_range", 1.0, 10.0, 5.0)`). Python's built-in `tkinter` library is a solid choice for this task. As the standard GUI toolkit, it is readily available and well-documented [39] [332]. Numerous examples demonstrate how to create dynamic widgets and bind their values to Python variables [150] [151]. For instance, one can create a slider and store a corresponding `DoubleVar` or `IntVar` object, allowing the script to read the configured value after the GUI is closed [43] [44]. Dropdown menus can be created using `OptionMenu` and populated dynamically based on a list of choices provided in the script [37] [41]. While `tkinter` widgets can appear dated, they are functional and lightweight. For a more modern aesthetic, frameworks like `PySide6` or `CustomTkinter` offer visually appealing components with less effort [147] [149]. The key is to design a simple convention for the script to expose its variables, which the host application can then parse to construct the GUI layout on the fly [322]. This ensures that the configuration panel is always tailored to the specific needs of the loaded script.

During runtime, the requirements shift dramatically. The overlay must be transparent, have minimal impact on system resources, and ideally not intercept mouse clicks, so it

does not block interaction with the game. Standard windowing toolkits like `Tkinter` or `PyQt` are unsuitable for this purpose as they are designed for traditional desktop applications and introduce significant rendering overhead. The definitive solution for this use case is an immediate-mode GUI (IMGUI) library like `Dear ImGui`. Originally developed for debugging and profiling in-game editors, Dear ImGui is designed to be exceptionally fast and lightweight [256][296]. It does not manage a persistent widget hierarchy; instead, it generates optimized vertex buffers every frame based on function calls within the application's render loop [296]. These buffers are then submitted to the underlying graphics API (DirectX, Vulkan, OpenGL) for rendering directly on top of the game画面. This tight integration with the rendering pipeline results in virtually zero performance overhead, making it perfect for a high-performance overlay [88].

Numerous guides and repositories detail how to implement an ImGui overlay for games, particularly with DirectX 11 [120][121][168]. Creating a transparent, click-through window is a standard capability of ImGui and can be achieved with a few settings. This ensures that the overlay's buttons (Play, Pause, Quit) are visible but do not interfere with the player's ability to interact with the game behind them [294][354]. The overlay would display essential runtime information and controls, such as the current status, action logs, or debug visuals from the vision module. The entire UI is rendered on-the-fly each frame, which simplifies state management compared to retained-mode GUIs [141]. Python bindings for ImGui, such as `imgui-bundle`, make it possible to leverage this powerful technology from a Python application, combining the logic of the bot with the high-performance rendering of ImGui [169][255]. The combination of a dynamic `tkinter`-based config panel and a high-performance ImGui overlay provides a complete and effective UI solution that meets all the user's requirements.

State Management and Execution Framework

The operational logic of the game bot is governed by the custom script, which dictates a sequence of actions based on sensory input from the computer vision system. To execute this logic reliably, the application must implement a robust state management framework that can handle the bot's lifecycle (running, paused, stopped) and coordinate the interactions between its various asynchronous components. A failure to properly manage state is a common source of bugs in automation systems, leading to bots that get stuck in a particular state or fail to recover gracefully from errors [320][321]. The design should draw

inspiration from established patterns in process automation and finite state machines to ensure predictable and resilient behavior [62] [220].

At its core, the bot operates as a continuous loop, often referred to as the main execution thread or a worker thread. In each iteration of this loop, the bot performs a series of steps: 1) Request a new screenshot from the capture module, 2) Process the screenshot using the vision pipeline (object and text recognition), 3) Pass the recognition results to the script engine for evaluation, and 4) Execute any actions returned by the script engine (e.g., press a key, wait for a duration). This cycle repeats indefinitely until a termination condition is met. The script itself would be structured using the requested control flow statements like `if/else` and `switch` to make decisions based on the vision results [280]. For example: `if health_bar_detected() > 0.8 then use_potion() end`.

Central to this loop is the concept of state. The bot must maintain a clear state—such as `RUNNING`, `PAUSED`, or `STOPPED`—that dictates the behavior of the main loop. When the user clicks the "Pause" button from the overlay, the application should atomically update this state flag. The main execution loop would then check this flag on each iteration and, if the state is `PAUSED`, simply skip the processing and action phases, perhaps updating a debug log to indicate that it is waiting. This prevents the bot from consuming CPU cycles unnecessarily while paused. The "Play" button would change the state back to `RUNNING`. The "Quit" button would trigger a graceful shutdown, stopping the capture thread, the vision workers, and exiting the main loop. The "Restart" function is particularly important for usability; it should cleanly reset the bot's internal state and re-initialize the script engine, ensuring that any lingering variables or conditions from a previous run do not affect the new session [215] [319].

To maintain responsiveness, especially during the lengthy periods of waiting for delays or performing vision tasks, all long-running operations must be offloaded from the main thread. This is a fundamental principle of GUI and real-time application development. The screen capture should run in its own thread, feeding a queue. The vision processing (template matching, YOLO inference, OCR) should be handled by a thread pool, with each worker pulling frames from the queue and placing results onto another queue. The script execution, which involves interpreting the DSL and calling external functions, should happen in yet another thread, pulling results from the vision queue. This multi-threaded, message-passing architecture ensures that no single component blocks the entire system. The main thread, responsible for rendering the ImGui overlay, remains free to respond to user input (play/pause/quit) instantly. This separation of concerns is a hallmark of well-designed automation frameworks and is essential for building a stable and reliable bot [47] [266]. Best practices from broader automation literature, such as having

clear coding standards and managing state transitions carefully, should be adopted to ensure the system is maintainable and understandable [61] [62].

Synthesis and Strategic Recommendations

The development of a general-purpose PC game bot application as specified by the research goal is a technically ambitious but achievable endeavor. The synthesis of the analyzed materials points towards a modular, high-performance architecture that prioritizes real-time responsiveness, user extensibility, and system security. The success of this project hinges on making informed decisions at each stage of the architecture, balancing the competing demands of speed, accuracy, and ease of use. The final recommendations provide a concrete, actionable blueprint for constructing the desired application.

First, the foundation of the entire system must be a high-performance screen capture engine. The evidence overwhelmingly favors a Windows-centric approach, utilizing a library built upon the Desktop Duplication API, such as `DXcam` or `D3DShot` [31] [164]. These libraries provide the necessary throughput (over 240 FPS) and stability to feed a real-time vision pipeline without dropping frames, which is critical for the bot's effectiveness. This choice establishes Windows as the primary development and target platform.

Second, the computer vision module should be designed as a hybrid pipeline. It must incorporate both a fast, multi-template matching algorithm for identifying objects from user-provided image samples and a robust, lightweight deep learning model like YOLOv10n or YOLOv12n for more complex object detection [109] [157]. For text recognition, PaddleOCR stands out as the superior choice due to its exceptional balance of speed, accuracy, and CPU efficiency, making it ideal for real-time scene text extraction [50] [51]. This dual-path vision system allows the bot to intelligently select the most appropriate and efficient method for each detection task.

Third, the custom scripting engine is a central pillar of the application's extensibility. While embedding a full language like Python offers power, it introduces severe security risks that are difficult to mitigate without significant complexity. The most strategic approach is to design and implement a custom, domain-specific language (DSL) tailored specifically for botting tasks. This can be accomplished using Python's built-in parsing and abstract syntax tree (AST) modules, providing a secure, simple, and highly controllable

execution environment that perfectly aligns with the user's request for a "Lua-like" but safe scripting interface [103][179].

Fourth, the user interface must be architected for two distinct modes. Before script execution, a dynamic configuration GUI should be generated using a framework like `Tkinter` or `PySide6`, reading variable definitions directly from the script to create sliders, toggles, and dropdowns [150][343]. During runtime, the overlay must be minimalist and non-intrusive. The recommended technology for this is `Dear ImGui`, an immediate-mode GUI library that integrates directly with the graphics API (e.g., DirectX) to render controls with near-zero performance overhead, ensuring it does not interfere with gameplay [88] [296].

Finally, the entire application must be underpinned by a robust state management framework built on a multi-threaded, producer-consumer architecture. This ensures that screen capture, vision processing, and script execution run concurrently without blocking one another, while a centralized state manager handles the bot's lifecycle (play, pause, stop, restart) reliably [62] [214]. Adherence to these architectural principles—prioritizing performance at the lowest level, building a versatile vision system, ensuring security through a custom DSL, designing a responsive UI, and managing state rigorously—will enable the creation of a powerful, extensible, and reliable general-purpose game automation tool that fulfills all aspects of the user's research goal.

Reference

1. microsoft/botframework-sdk: Bot Framework provides the ... https://github.com/microsoft/botframework-sdk
2. Bot Framework Web Chat https://github.com/microsoft/botframework-webchat
3. game-script · GitHub Topics https://github.com/topics/game-script
4. microsoft/botbuilder-python: The Microsoft Bot Framework ... https://github.com/microsoft/botbuilder-python
5. How do I get started with developing a basic AI chatbot ... https://github.com/orgs/community/discussions/143385
6. Game Agent Framework. Helping you create AIs / Bots ... https://github.com/SerpentAI/SerpentAI

7. Create a bot with the Bot Framework SDK https://learn.microsoft.com/en-us/azure/bot-service/bot-service-quickstart-create-bot?view=azure-bot-service-4.0
8. Trending Open-Source Github Projects : Fish Speech, AstrBot ... https://www.youtube.com/watch?v=jWfDo1sQOqE&vl=en-US
9. python-chatbot https://github.com/topics/python-chatbot
10. How can I sandbox Python in pure Python? https://stackoverflow.com/questions/3068139/how-can-i-sandbox-python-in-pure-python
11. Running Untrusted Python Code - Andrew Healey https://healeycodes.com/running-untrusted-python-code
12. Create a Python Sandbox for Agents to Run Code https://www.youtube.com/watch?v=bm6jegefGyY
13. Running Python code in a sandbox with MicroPython and ... https://simonwillison.net/2026/Jun/6/micropython-in-a-sandbox/
14. python - Best practices for execution of untrusted code https://softwareengineering.stackexchange.com/questions/191623/best-practices-for-execution-of-untrusted-code
15. Setting Up a Secure Python Sandbox for LLM Agents - dida.do https://dida.do/blog/setting-up-a-secure-python-sandbox-for-llm-agents
16. How can I run an untrusted Python script safely (ie Sandbox) https://wiki.python.org/moin/Asking(20)for(20)Help(2f)How(20)can(20)I(20)run(20)an(20)untrusted(20)Python(20)script(20)safely(2028)i(2e)e(2e20)Sandbox(29).html
17. Sandboxed Python Environment https://www.danielcorin.com/posts/2024/sandboxed-python-env/
18. openedx/codejail: Secure code execution https://github.com/openedx/codejail
19. Other easily embeddable languages like Lua. https://www.reddit.com/r/ProgrammingLanguages/comments/h7u9v7/other_easily_embeddable_languages_like_lua/
20. embedded-scripting-languages | Lobsters https://lobste.rs/s/rctrmn/embedded_scripting_languages
21. With Lua and Python embeddable, is there a place for Basic? https://stackoverflow.com/questions/244138/with-lua-and-python-embeddable-is-there-a-place-for-basic
22. A list of embedded scripting languages https://github.com/dbohdan/embedded-scripting-languages
23. Scripting Languages https://arewegameyet.rs/ecosystem/scripting/

24. programming - First language designed to be embedded in ... https://retrocomputing.stackexchange.com/questions/8146/first-language-designed-to-be-embedded-in-another-program
25. The smallest embeddable scripting language, part 1 https://log.schemescape.com/posts/static-site-generators/smallest-scripting-language.html
26. This is an unpopular opinion, but I wish Lua had won ... https://news.ycombinator.com/item?id=16404562
27. Why is Lua not more popular as an embedded scripting ... https://www.quora.com/Why-is-Lua-not-more-popular-as-an-embedded-scripting-language
28. How to Capture Desktop Screen with DXcam in Python https://screenshotone.com/blog/dxcam-python-screenshots/
29. FAST Screenshots in Python for Computer Vision: mss vs. PIL vs ... https://www.youtube.com/watch?v=SWgQNWf1ICA
30. Fastest way to take a screenshot with python on windows https://stackoverflow.com/questions/3586046/fastest-way-to-take-a-screenshot-with-python-on-windows
31. ra1nty/DXcam: A Python high-performance screen capture library ... https://github.com/ra1nty/DXcam
32. D3DShot: Extremely Fast Screen Capture on Windows with the ... https://www.reddit.com/r/Python/comments/b9fb2s/d3dshot_extremely_fast_screen_capture_on_windows/
33. Python - Fast Screen Capture https://kylefu.me/2023/02/18/python-fast-screen-capture.html
34. YOLOv8 vs. EfficientDet: A Deep Dive into the Future of ... https://medium.com/suitable-ryans-thoughts/yolov8-vs-efficientdet-a-deep-dive-into-the-future-of-real-time-object-detection-code-at-bottom-86513de2f69d
35. Benchmarking Deep Learning Models for Object Detection ... https://arxiv.org/html/2409.16808v1
36. Benchmarking YOLOv8–YOLOv12 for Real-Time Object ... https://www.preprints.org/manuscript/202605.0936
37. Creating a Dropdown Menu in Python Tkinter: A Step-by-Step ... https://www.youtube.com/watch?v=wmVaFlTFB_I
38. Python tkinter: create a dynamic dropdown menu and call ... https://stackoverflow.com/questions/54464292/python-tkinter-create-a-dynamic-dropdown-menu-and-call-different-actions-after
39. The ultimate introduction to modern GUIs in Python [with tkinter] https://www.youtube.com/watch?v=mop6g-c5HEY

40. Running another script from button click · Issue #2050 https://github.com/PySimpleGUI/PySimpleGUI/issues/2050
41. 16. Dynamic Dropdown in Tkinter (Python) https://www.youtube.com/watch?v=suKySOoQFSs
42. List of Dynamic Python GUI's ? : r/learnpython https://www.reddit.com/r/learnpython/comments/xeve6j/list_of_dynamic_python_guis/
43. How to access values from a list of dynamically generated ... https://community.plotly.com/t/how-to-access-values-from-a-list-of-dynamically-generated-sliders/20125
44. Stylish Python CustomTkinter GUIs #5 | by Strive to Develop https://medium.com/@BetterEverything/sliders-connected-to-functions-stylish-python-customtkinter-guis-5-6ceb4f5f31a
45. Interactive Sliders with Python Tkinter | GUI Tutorial https://www.youtube.com/watch?v=aBU2G9v3sa0
46. Game Automation with YOLOv8: Python Bot Tutorial https://www.youtube.com/watch?v=PDzifxuEric
47. How To Bot with OpenCV - OpenCV Object Detection in Games #9 https://www.youtube.com/watch?v=FylRUEEWhCo
48. How To Build a Bot with OpenCV - LearnCodeByGaming.com https://learncodebygaming.com/blog/how-to-build-a-bot-with-opencv
49. OCR comparison: Tesseract versus EasyOCR vs PaddleOCR ... https://toon-beerten.medium.com/ocr-comparison-tesseract-versus-easyocr-vs-paddleocr-vs-mmocr-a362d9c79e66
50. [P] Choosing an OCR : r/MachineLearning https://www.reddit.com/r/MachineLearning/comments/i98wr6/p_choosing_an_ocr/
51. Paddle OCR vs Tesseract: Detailed OCR Comparison https://ironsoftware.com/csharp/ocr/blog/compare-to-other-components/paddle-ocr-vs-tesseract/
52. PaddleOCR: analysis, benefits and open source alternatives https://www.koncile.ai/en/ressources/paddleocr-analyse-avantages-alternatives-open-source
53. Technical Analysis of Modern Non-LLM OCR Engines https://intuitionlabs.ai/articles/non-llm-ocr-technologies
54. substitute PaddleOCR for Tesseract-OCR · Issue #10232 https://github.com/siyuan-note/siyuan/issues/10232
55. PaddleOCR vs Tesseract vs EasyOCR: OCR Model ... https://gigagpu.com/paddleocr-vs-tesseract-vs-easyocr/
56. multi-template-matching/MultiTemplateMatching-Python https://github.com/multi-template-matching/MultiTemplateMatching-Python

57. Multi-scale Template Matching using Python and OpenCV https://pyimagesearch.com/2015/01/26/multi-scale-template-matching-using-python-opencv/
58. Multi-Template Matching for object-detection https://multi-template-matching.github.io/Multi-Template-Matching/
59. Automating basic tasks in games with OpenCV and Python https://www.tautvidas.com/blog/2018/02/automating-basic-tasks-in-games-with-opencv-and-python/
60. Unreal Debugging Tools I Wish I knew earlier! https://www.youtube.com/watch?v=XC_ntVpHg80
61. How to dynamically change Game Mode/State? https://www.reddit.com/r/unrealengine/comments/e99uzr/how_to_dynamically_change_game_modestate/
62. The State of Process Automation- Goodbye, Scripted Bots https://medium.com/@adnanmasood/the-state-of-process-automation-goodbye-scripted-bots-hello-ai-co-workers-2caa44ef7b4f
63. Is there a more efficient way to handle state transitions? https://www.facebook.com/groups/godotengine/posts/2235355726601009/
64. Gameplay Ability System - Best Practices for Setup https://dev.epicgames.com/community/learning/tutorials/DPpd/unreal-engine-gameplay-ability-system-best-practices-for-setup
65. Activities - UI Automation Modern Project Settings https://docs.uipath.com/activities/other/latest/ui-automation/project-settings-ui-automation-next
66. Using Unreal Automation at Riot Games: Make Unreal Do the ... https://www.youtube.com/watch?v=3ftOkc-cA7U
67. Game Testing Frameworks & Test Automation Guide (2026) https://generalistprogrammer.com/tutorials/game-testing-frameworks-complete-automation-guide-2025
68. Top 15 UI Test Automation Best Practices https://www.blazemeter.com/blog/ui-test-automation
69. Automated Web UI Testing: Best Practices, Challenges, & ... https://www.parasoft.com/blog/automated-web-ui-testing-best-practices-challenges-tools/
70. Object detection lite : Template Matching https://medium.com/@BH_Chinmay/object-detection-lite-template-matching-c9af77517f6c
71. 118 - Object detection by template matching https://www.youtube.com/watch?v=P5FTEryiTl4
72. Image based Template matching services : r/computervision https://www.reddit.com/r/computervision/comments/15tt1d0/image_based_template_matching_services/

73. Template-based versus Feature-based Template Matching https://medium.datadriveninvestor.com/template-based-versus-feature-based-template-matching-e6e77b2a3b3a
74. OpenCV Template Matching (cv2.matchTemplate) https://pyimagesearch.com/2021/03/22/opencv-template-matching-cv2-matchtemplate/
75. Comparison of FPS difference between template matching ... https://www.researchgate.net/figure/Comparison-of-FPS-difference-between-template-matching-method-and-using-YOLO-to-process_fig5_380729550
76. OpenCV template matching or feature detection to correctly ... https://stackoverflow.com/questions/48081791/opencv-template-matching-or-feature-detection-to-correctly-categorize-types-of-f
77. DirectX 11 vs Vulkan: which is best for Baldur's Gate 3? https://www.reddit.com/r/Games/comments/15lmg6n/directx_11_vs_vulkan_which_is_best_for_baldurs/
78. Rewriting My DirectX Game Engine to Vulkan https://www.youtube.com/watch?v=_KHHKedYNOc
79. It's Not About The API - Fast, Flexible, and Simple Rendering ... https://www.youtube.com/watch?v=7bSzp-QildA
80. Baldur's Gate 3 - DirectX 11 or Vulkan??? https://steamcommunity.com/app/1086940/discussions/0/6045572169632538573/?ctp=3
81. Vulkan or DirectX11 with Patch 7? https://forums.larian.com/ubbthreads.php?ubb=showflat&Number=948828
82. Baldurs Gate 3 - Vulkan vs DX11 Performance (2026 Update) https://www.youtube.com/watch?v=PYsIAPrUid4
83. DirectX 11 or Vulkan for non-DirectX 12 games? https://www.facebook.com/groups/pcbuilderandsetups/posts/1506889971111038/
84. No Graphics API https://news.ycombinator.com/item?id=46293062
85. DirectX 11 vs Vulkan: which is best for Baldur's Gate 3? https://www.digitalfoundry.net/articles/digitalfoundry-2023-baldurs-gate-3-directx-11-vs-vulkan-which-is-best
86. Fastest screenshot library python/improve performance of ... https://stackoverflow.com/questions/54540307/fastest-screenshot-library-python-improve-performance-of-mss-package
87. How to Capture Screen with Python (Very Fast!!!) https://www.youtube.com/watch?v=4kYrIwSZRMg
88. pyimgui/pyimgui: Cython-based Python bindings for dear ... https://github.com/pyimgui/pyimgui
89. imgui https://pypi.org/project/imgui/

90. Looking for lightweight OCR to read display, preferably ... https://www.reddit.com/r/LocalLLaMA/comments/1d8wcho/looking_for_lightweight_ocr_to_read_display/
91. Apps on Instagram: "testing an FPS (First-Person Shooter ... https://www.instagram.com/reel/DGYV3ZqPXGG/
92. Correcting Tesseract OCR errors with LLMs https://news.ycombinator.com/item?id=41203306
93. bobeff/open-source-engines: A list of open source game engines. https://github.com/bobeff/open-source-engines
94. game-scripting · GitHub Topics https://github.com/topics/game-scripting?o=asc&s=stars
95. Overload-Technologies/Overload: 3D game engine with lua scripting https://github.com/Overload-Technologies/Overload
96. game-automation https://github.com/topics/game-automation
97. lua-scripting · GitHub Topics https://github.com/topics/lua-scripting?l=python&o=desc&s=forks
98. open-source-game https://github.com/topics/open-source-game?l=lua&o=desc&s=stars
99. LewisJEllis/awesome-lua: A curated list of quality Lua packages and ... https://github.com/lewisjellis/awesome-lua
100. AI Coding Tools for Video Game Development: A First-Principles ... https://chierhu.medium.com/ai-coding-tools-for-video-game-development-a-first-principles-analysis-of-what-actually-works-90dfa10edd13
101. Embedding a Low Performance Scripting Language in ... https://stackoverflow.com/questions/5099043/embedding-a-low-performance-scripting-language-in-python
102. Embedding Lua vs Python https://eev.ee/blog/2016/04/30/embedding-lua-vs-python/
103. Why I Built My Own DSL in Python (And How You Can Too) https://medium.com/illumination/why-i-built-my-own-dsl-in-python-and-how-you-can-too-9796a55606a0
104. python - Alternative Scripting Language to Lua? https://softwareengineering.stackexchange.com/questions/191380/alternative-scripting-language-to-lua
105. Lua: an extensible embedded language https://www.lua.org/ddj.html
106. Unleash the Power of IoT with the Lua language: A Step-by ... https://www.youtube.com/watch?v=XkVQYJ-z7JA
107. For a live camera video feed, is orb more suitable or yolo for logo ... https://forum.opencv.org/t/for-a-live-camera-video-feed-is-orb-more-suitable-or-yolo-for-logo-detection/17863

108. An Overview of YOLO26, YOLO11, YOLOv8, and YOLOv5 Object ... https://arxiv.org/html/2510.09653v3
109. What's the fastest object detection model? : r/computervision https://www.reddit.com/r/computervision/comments/1h35tpi/whats_the_fastest_object_detection_model/
110. YOLOv26: An Object Detector Built for Real-Time Deployment https://learnopencv.com/yolov26-real-time-deployment/
111. Best Object Detection Models 2026: RF-DETR, YOLOv12 & Beyond https://blog.roboflow.com/best-object-detection-models/
112. Review of YOLOv10 vs YOLOv8: Model Size, Performance Metrics, ... https://www.dfrobot.com/blog-13998.html?srsltid=AfmBOoqRBPEQK_msQDLdKSacafM5j4mmMUQmEu482593ws2TV_lpw18Z
113. Want to speed up template matching in OpenCV python https://stackoverflow.com/questions/65464727/want-to-speed-up-template-matching-in-opencv-python
114. Model Prediction with Ultralytics YOLO https://docs.ultralytics.com/modes/predict
115. LightGlue: Local Feature Matching at Light Speed https://github.com/cvg/lightglue
116. Fullscreen doesn't capture as expected with fullscreen game https://github.com/BoboTiG/python-mss/issues/28
117. PSA: Disabling full screen optimization in some games can ... https://www.reddit.com/r/pcgaming/comments/11jsqc7/psa_disabling_full_screen_optimization_in_some/
118. (windows) allow desktop screen capture to fail gracefully ... https://github.com/awawa-dev/HyperHDR/issues/1041
119. Transparency issues with ImGUI #8354 https://github.com/ocornut/imgui/issues/8354
120. Dear IMGUI and DirectX 12 Overlay ... https://stackoverflow.com/questions/63615630/dear-imgui-and-directx-12-overlay-dxgi-error-invalid-call
121. Universal C++ Internal DX9 ImGui Overlay https://guidedhacking.com/threads/universal-c-internal-dx9-imgui-overlay.19735/
122. Dx11 Overlay Transparency Issue #892 - ocornut/imgui https://github.com/ocornut/imgui/issues/892
123. Improving the latency of an OCR-based system https://medium.com/red-buffer/improving-the-latency-of-an-ocr-based-system-196f5badace9
124. Tesseract vs PaddleOCR vs dots.ocr. https://codesota.com/ocr/paddleocr-vs-tesseract
125. tips for improving Tesseract accuracy and speed... https://groups.google.com/g/tesseract-ocr/c/enwft4qSDfE

126. dxcam https://pypi.org/project/dxcam/0.1.0.dev2/
127. 19 Optimizing with Dxcam https://www.youtube.com/watch?v=YBm2a3v9vH8
128. The 0.9B OCR Model That Beats Gemini? (GLM-OCR) | Benchmarks ... https://www.youtube.com/watch?v=_KCXD8vFIYM
129. Real time OCR of Unstructured Text | by Ruvinda Dhambarage - Medium https://ruvid.medium.com/real-time-ocr-of-unstructured-text-7cbbb3162e94
130. python - Real time OCR lag https://stackoverflow.com/questions/73396902/real-time-ocr-lag
131. Improving Text Detection Speed with OpenCV and GPUs https://pyimagesearch.com/2022/03/14/improving-text-detection-speed-with-opencv-and-gpus/
132. Vulkanised 2026: Vulkan Now and Then (for Hobbyists) https://www.youtube.com/watch?v=EshkHyYxb3A
133. What to use for In-Game overlay : r/AskProgramming https://www.reddit.com/r/AskProgramming/comments/9b8jnv/what_to_use_for_ingame_overlay/
134. jackun/vulkan-overlay-layer https://github.com/jackun/vulkan-overlay-layer
135. How to Vulkan in 2026 - How to Vulkan https://www.howtovulkan.com/
136. From Zero Python to a Desktop Overlay App in One Weekend https://www.wilfridwong.com/2026/04/04/zero-python-desktop-overlay-weekend/
137. Creating a transparent GUI overlay on top of a Full screen ... https://stackoverflow.com/questions/58759482/creating-a-transparent-gui-overlay-on-top-of-a-full-screen-game-in-python
138. Vulkan Introduces Roadmap 2026 and New Descriptor ... https://www.khronos.org/blog/vulkan-introduces-roadmap-2026-and-new-descriptor-heap-extension
139. Path Traced Hair & Skin in AAA Games using Vulkan https://www.youtube.com/watch?v=mTR8NxkYzw0
140. How to Actually Build Mobile Apps with AI in 2026 | A ... https://www.youtube.com/watch?v=Q7AYc2kECDI
141. How to manage state in a complex UI? : r/roguelikedev https://www.reddit.com/r/roguelikedev/comments/9u6ttj/how_to_manage_state_in_a_complex_ui/
142. What's new from Config 2026 https://help.figma.com/hc/en-us/articles/39582753756695-What-s-new-from-Config-2026
143. The Developer's Guide to Generative UI in 2026 | Blog https://www.copilotkit.ai/blog/the-developer-s-guide-to-generative-ui-in-2026
144. Replay Recap: Spring 2026 Temporal Product Updates https://www.youtube.com/watch?v=RL8AkFd7u9o

145. Best Automated UI Testing Tools Update July 2026 | Guide https://www.skyvern.com/blog/automated-ui-testing-tools-guide/
146. Using Robots in Oracle Integration 3 https://docs.oracle.com/en/cloud/paas/application-integration/robotic-automation/using-robots-oracle-integration-3.pdf
147. Nothing Can Stop you Now from Creating Awesome GUI | Python https://www.youtube.com/watch?v=F7YQaUcDsRk
148. Dynamic GUI in Python? https://stackoverflow.com/questions/16570631/dynamic-gui-in-python
149. Which Python GUI library should you use in 2026? https://www.pythonguis.com/faq/which-python-gui-library/
150. Python Example - Dynamic with GUI https://help.syscad.net/Python_Example_-_Dynamic_with_GUI
151. cahidenes/dynamic_variables: Change python variables during ... https://github.com/cahidenes/dynamic_variables
152. RapidAI/RapidOCR: 📄 Awesome OCR multiple programing ... https://github.com/rapidai/rapidocr
153. Robust Scene Text Detection and Recognition: Inference ... https://developer.nvidia.com/blog/robust-scene-text-detection-and-recognition-inference-optimization/
154. Best Open Source OCR Tools & Models in 2026 https://unstract.com/blog/best-opensource-ocr-tools/
155. I tested 6 Python OCR libraries on the same invoice. https://codesota.com/ocr/best-for-python
156. YOLOv9 vs YOLOv10 vs YOLOv11 vs YOLOv12 FPS & ... https://www.facebook.com/Pyresearch/posts/yolov9-vs-yolov10-vs-yolov11-vs-yolov12-fps-performance-comparison/1081718790635069/
157. Benchmarking Lightweight YOLO Object Detectors for Real ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12526732/
158. GitHub - HarishJ001/Game-Automation: A simple yet powerful ... https://github.com/HarishJ001/Game-Automation
159. OpenCV Object Detection in Games - LearnCodeByGaming.com https://learncodebygaming.com/blog/tutorial/opencv-object-detection-in-games
160. Python game bot project structure — Grokipedia https://grokipedia.com/page/Python_game_bot_project_structure
161. Top 10 Python pyautogui Projects | LibHunt https://www.libhunt.com/l/python/topic/pyautogui

162. Game Bot Development with Python - CodePal https://codepal.ai/chat/query/QEbaaJg1/build-game-bot-with-python
163. Python Automation Project for Beginners: Build an AI Dino ... https://www.youtube.com/watch?v=c4zrEKQkw1c
164. SerpentAI/D3DShot https://github.com/SerpentAI/D3DShot
165. Desktop Duplication API skip frame issue and slowness issue https://forums.developer.nvidia.com/t/desktop-duplication-api-skip-frame-issue-and-slowness-issue/182492
166. Screen Capture with DXGI https://gamedev.net/forums/topic/642194-screen-capture-with-dxgi/
167. Windows Desktop Duplication https://lib.rs/crates/win_desktop_duplication
168. Dx11 Overlay With ImGUI https://www.unknowncheats.me/forum/1565719-post1.html
169. imgui-bundle https://pypi.org/project/imgui-bundle/0.7.2/
170. RootKit-Org/BetterCam https://github.com/RootKit-Org/BetterCam
171. Windows Graphics Capture vs DXGI Desktop Duplication https://obsproject.com/forum/threads/windows-graphics-capture-vs-dxgi-desktop-duplication.149320/
172. YOLOv10: NMS-Free Object Detection https://docs.ultralytics.com/models/yolov10
173. Review of YOLOv10 vs YOLOv8: Model Size, Performance Metrics, ... https://www.dfrobot.com/blog-13998.html?srsltid=AfmBOorDt8MUJJlGKt_c5er9514vDv3JizyXdHb-PVgRTxBIGoXHZFVq
174. Benchmarking YOLO-Based Deep Learning Models for Real-Time ... https://papers.ssrn.com/sol3/Delivery.cfm/dc841ca6-4967-4552-ac23-de4e96d4906a-MECA.pdf?abstractid=5598062&mirid=1
175. I don't understand Lua, why it's good, why it's used in embedded ... https://www.reddit.com/r/learnprogramming/comments/1ds0z21/i_dont_understand_lua_why_its_good_why_its_used/
176. Stay away from Lua and use Python, Go, Node, Rust or even C in new projects. ... https://news.ycombinator.com/item?id=18334407
177. Lua, Python and all the others: When to use which programming language? https://exasol.my.site.com/s/article/Lua-Python-and-all-the-others-When-to-use-which-programming-language
178. Why Python and not Lua? [closed] https://softwareengineering.stackexchange.com/questions/66590/why-python-and-not-lua
179. Build Your Own Domain Specific Language in Python With textX https://www.youtube.com/watch?v=9JgIL1XTYRo

180. Python + Qt -> 2D overlay over other full screen app? https://stackoverflow.com/questions/12334217/python-qt-2d-overlay-over-other-full-screen-app
181. Successor to opengl ? : r/GraphicsProgramming https://www.reddit.com/r/GraphicsProgramming/comments/1t6ba1h/successor_to_opengl/
182. game-overlay-sdk https://pypi.org/project/game-overlay-sdk/
183. Transparent Overlays download https://sourceforge.net/projects/transparent-overlays/
184. Keeping Python Application Always on Top (Even During Gaming) https://learn.microsoft.com/en-us/answers/questions/3918271/keeping-python-application-always-on-top-(even-dur
185. Asdf Overlay – High performance in-game overlay library for Windows https://news.ycombinator.com/item?id=44138775
186. Vulkanised 2026: Power Friendly Vulkan - QCOM Extensions https://www.youtube.com/watch?v=_BwCwNd2BHc
187. 7 Python Automation Scripts That Save 10+ Hours [2026] https://tech-insider.org/python-automation-tutorial-tasks-2026/
188. How I Built a Python Automation Bot That Manages My ... https://python.plainenglish.io/how-i-built-a-python-automation-bot-that-manages-my-projects-b073a14620b7
189. Python Automation Explained: Real-World Use Cases for ... https://www.youtube.com/watch?v=CjoaPBzZYBc
190. Best Practices for VFX Pipeline Automation with Python https://dev.to/teevirta/best-practices-for-vfx-pipeline-automation-with-python-39m2
191. Chatbot UI Design Patterns and Best Practices 2026 https://fuselabcreative.com/chatbot-interface-design-guide/
192. Building a Python-Powered Automation Framework That ... https://medium.com/illumination/building-a-python-powered-automation-framework-that-took-my-workflow-from-10-hours-to-10-minutes-b30da75ffff6
193. Automate Tasks with Python: Technical Guide for 2026 Logic https://zignuts.com/blog/automate-tasks-with-python
194. Python Automation for AI Tasks - Complete Guide https://zenvanriel.com/ai-engineer-blog/python-automation-ai-tasks/
195. YouTube https://www.youtube.com/watch?v=KwHWyDrO-Gw
196. Your First PyAutoGUI Game Bot: PyAutoGUI Video Game Bot ... https://www.youtube.com/watch?v=NaZTtUmE990
197. Used Pyscreenshot, PyAutoGui and OpenCV-python to ... https://www.reddit.com/r/Python/comments/c7h3ze/used_pyscreenshot_pyautogui_and_opencvpython_to/

198. Automate Your Bot Script Creation: PyAutoGUI Video Game ... https://www.youtube.com/watch?v=ixwVSsKydIY

199. steve1316/granblue-automation-pyautogui ... https://github.com/steve1316/granblue-automation-pyautogui

200. OpenCV + PyAutoGui custom scripts viable? https://www.reddit.com/r/RunescapeBotting/comments/1qipf59/opencv_pyautogui_custom_scripts_viable/

201. Rudimentary Computer Vision Techniques for Python Bot. https://stackoverflow.com/questions/15685894/rudimentary-computer-vision-techniques-for-python-bot

202. Made a simple bot for a game using PyAutoGui and PIL. : r ... https://www.reddit.com/r/Python/comments/ccc783/made_a_simple_bot_for_a_game_using_pyautogui_and/

203. What do I think about Lua after shipping a project with ... https://blog.luden.io/what-do-i-think-about-lua-after-shipping-a-project-with-60-000-lines-of-code-bf72a1328733

204. What scripting language is easy to integrate for game ... https://www.facebook.com/groups/IndieGameDevs/posts/10153538510731573/

205. Lua Game Engines in 2025 https://www.youtube.com/watch?v=hBvr3d2fg7Y

206. lua · GitHub Topics https://github.com/topics/lua?l=go

207. Imagine Making game engine With Lua https://www.reddit.com/r/lua/comments/1s5njb7/imagine_making_game_engine_with_lua/

208. Scripting behaviour in a game engine - A small example https://snoozetime.github.io/2019/01/29/scripting-gameengine.html

209. Review of YOLOv10 vs YOLOv8: Model Size, Performance Metrics, ... https://www.dfrobot.com/blog-13998.html?srsltid=AfmBOootzreS7xkqbO-wOC3W-5TH6eBnRu6S91SSvukyOVYq-8-k7Mly

210. RealTime-OCR [Version1] https://github.com/nathanaday/RealTime-OCR

211. NVIDIA-AI-IOT/scene-text-recognition https://github.com/NVIDIA-AI-IOT/scene-text-recognition

212. ruslanmv/BOT-MMORPG-AI https://github.com/ruslanmv/BOT-MMORPG-AI

213. Pytesseract is very slow for real time OCR, any way to ... https://stackoverflow.com/questions/66334737/pytesseract-is-very-slow-for-real-time-ocr-any-way-to-optimise-my-code

214. real-time-processing https://github.com/topics/real-time-processing?l=python

215. Restarting Python Automation in 2026 - Ryan Scott Brown https://rsb.io/posts/python-automation-starter-guide/

216. Activities - v24.10 https://docs.uipath.com/activities/other/latest/ui-automation/release-notes-uipath-uiautomation-activities-v24-10

217. The Best AI Automation Stack to Learn in 2026 https://www.youtube.com/watch?v=omU3zR3K7-U&vl=en

218. Python 2026 Significant Changes Guide https://learn.microsoft.com/en-us/agent-framework/support/upgrade/python-2026-significant-changes

219. UiPath vs. Python https://www.reddit.com/r/UiPath/comments/gsacrx/uipath_vs_python/

220. How to python transitions for robotic applications / ... https://stackoverflow.com/questions/53006994/how-to-python-transitions-for-robotic-applications-embedded-systems

221. Python Automation Crash Course (2026): Automate Your Life ... https://www.youtube.com/watch?v=4AEsblWWdQg

222. Creating a transparent overlay with qt - python https://stackoverflow.com/questions/25950049/creating-a-transparent-overlay-with-qt

223. Possible to create a full screen overlay that catches clicks, ... https://users.rust-lang.org/t/possible-to-create-a-full-screen-overlay-that-catches-clicks-but-does-not-come-into-focus/75567

224. does Vulkan not actually run in fullscreen? : r/Rainbow6TTS https://www.reddit.com/r/Rainbow6TTS/comments/ewmicg/does_vulkan_not_actually_run_in_fullscreen/

225. Vulkan transparent window stopped working on Windows 11 https://github.com/godotengine/godot/issues/111513

226. Best Nvidia App Settings for MAX Performance in 2026 https://www.youtube.com/watch?v=4A0r1oJWFLE

227. Game bar overlay on Vulkan API - Baldur's Gate 3 https://steamcommunity.com/app/1086940/discussions/0/4289187252711723752/

228. How can I generate a full screen semi transparent window ... https://askubuntu.com/questions/1557564/how-can-i-generate-a-full-screen-semi-transparent-window-in-ubuntu

229. #10 Sliders: Building Modern GUIs using Python | Full Course ... https://www.youtube.com/watch?v=9hpVbATgUSs

230. The Python GUI Tool No One Told Me About (And Why It's ... https://python.plainenglish.io/the-python-gui-tool-no-one-told-me-about-and-why-its-a-game-changer-6e85686452b9

231. How to build a cross platform GUI with Python — Russell Keith ... https://www.youtube.com/watch?v=tuSnatBqItE

232. Python GUI with Tkinter - 9 - Creating Drop Down Menus https://www.youtube.com/watch?v=PSm-tq5M-Dc

233. What is the best GUI library for Python? https://www.reddit.com/r/Python/comments/wedvzi/what_is_the_best_gui_library_for_python/
234. EditableRangeSlider — Panel v1.9.3 https://panel.holoviz.org/reference/widgets/EditableRangeSlider.html
235. 10 Best Python GUI Frameworks in 2026 | by Megha Verma https://medium.com/predict/10-best-python-gui-frameworks-in-2026-c38a57c3cc94
236. Adding an AST phase for an interpreter : r/Compilers https://www.reddit.com/r/Compilers/comments/1pfm708/adding_an_ast_phase_for_an_interpreter/
237. DSL Operator – A different approach to DSLs - Ideas https://discuss.python.org/t/dsl-operator-a-different-approach-to-dsls/79874
238. Creating a Python Code Completer & More Abstract Syntax ... https://www.youtube.com/watch?v=CiyrZdmBHfE
239. Best [rbx] lua 5.1 abstract syntax tree? - Scripting Support https://devforum.roblox.com/t/best-rbx-lua-51-abstract-syntax-tree/195344
240. ASTEVAL: Minimal Python AST Evaluator https://lmfit.github.io/asteval/
241. #152: Understanding and using Python's AST https://www.youtube.com/watch?v=_yFpXIbnT1E
242. Building an interpreter: designing an AST https://stackoverflow.com/questions/43374737/building-an-interpreter-designing-an-ast
243. Lua parser and abstract syntax tree in Lua https://github.com/thenumbernine/lua-parser
244. Let's Build A Simple Interpreter. Part 11. https://ruslanspivak.com/lsbasi-part11/
245. Lua: The Little Language That Could : r/programming https://www.reddit.com/r/programming/comments/13u6t3a/lua_the_little_language_that_could/
246. Why do we need an embeddable programming language ... https://stackoverflow.com/questions/19176209/why-do-we-need-an-embeddable-programming-language-like-lua
247. The Lua Programming Language Beginner's Guide https://www.bmc.com/blogs/lua-programming-language/
248. Awesome list of resources for Game Engine Development. https://github.com/stevinz/awesome-game-engine-dev
249. Top 13 Scripting Languages You Should Pay Attention To https://kinsta.com/blog/scripting-languages/
250. API/module for making bots that play games? : r/Python https://www.reddit.com/r/Python/comments/kwjycz/apimodule_for_making_bots_that_play_games/

251. Get Started with a Great Beginner AI Project! https://learncodebygaming.com/blog/get-started-with-a-great-beginner-ai-project
252. Beating browser games with accessible Python (Part 2) https://medium.com/henngeblog/automating-victory-beating-browser-games-with-accessible-python-part-2-181134e8b589
253. How to Create a Video Game Bot with Python -- #pyautogui ... https://www.youtube.com/watch?v=01od80SMG3s
254. carlgwastaken/Overlay: An external ... https://github.com/carlgwastaken/Overlay
255. ImGui Bundle: (web) apps in pure Python https://www.reddit.com/r/Python/comments/1ltalg4/imgui_bundle_web_apps_in_pure_python/
256. Dear ImGui Bundle: (web) apps in pure Python with a single ... http://code-ballads.net/dear-imgui-bundle-build-real-time-python-web-applications-with-zero-fuss/
257. IMGUI EXTERNAL OVERLAY TUTORIAL C++ (2025) https://www.youtube.com/watch?v=s_7VPnu58-U
258. Windows - comfig docs https://docs.comfig.app/latest/os/windows/
259. How to Fix Stuttering in DirectX 12 Games (2026) – Easy Lag ... https://www.youtube.com/watch?v=soNAUed8ZEE
260. Justanormalpaster/BetterCam https://github.com/Justanormalpaster/BetterCam
261. PaddleOCR vs Tesseract vs IronOCR: Picking an OCR ... https://medium.com/@ahmad.sohail/paddleocr-vs-tesseract-vs-ironocr-picking-an-ocr-engine-for-net-10a24dc2802e
262. Image Pre-Processing Techniques for OCR https://medium.com/@TechforHumans/image-pre-processing-techniques-for-ocr-d231586c1230
263. The Importance of OCR Performance Optimization https://blog.filestack.com/importance-ocr-performance-optimization/
264. Optimizing OCR Performance: An Investigation into Image ... https://www.researchgate.net/publication/380137363_Optimizing_OCR_Performance_An_Investigation_into_Image_Preprocessing_Techniques
265. Advanced Optimization Techniques for Tesseract OCR in ... https://sparkco.ai/blog/advanced-optimization-techniques-for-tesseract-ocr-in-enterprises
266. Python Test Automation | Tools | Frameworks - Bhavik Jikadara https://bhavikjikadara.medium.com/python-test-automation-frameworks-and-tools-46c6353335d9
267. Comparative performance analysis of YOLOv8 small and ... https://www.researchgate.net/publication/

400412408_Comparative_performance_analysis_of_YOLOv8_small_and_larger_for_real-time_website-based_monitoring

268. A curated list of awesome Python frameworks, libraries ... https://github.com/Python-List/awesome-python-1
269. PO files — Packages not i18n-ed https://www.debian.org/international/l10n/po/todo
270. reSorcery - Resources to become a self-taught Genius. https://resorcery.pages.dev/
271. 平台和工具 - 架构师 https://architect.pub/book/export/html/36
272. archive-repos.txt https://archiveprogram.github.com/assets/img/archive-repos.txt
273. FYP_similartags/RerunKeming/allTags_test.txt at master https://github.com/lint0011/FYP_similartags/blob/master/RerunKeming/allTags_test.txt
274. Making your own programming language is easier than ... https://lisyarus.github.io/blog/posts/making-your-own-programming-language.html
275. Full Lua Crash Course 2.5 Hours 🖥️⌨️ Beginner's ... https://www.youtube.com/watch?v=zi-svfdcUc8
276. How do I write a simple interpreter in C? : r/C_Programming https://www.reddit.com/r/C_Programming/comments/1l5oqy5/how_do_i_write_a_simple_interpreter_in_c/
277. Building a Simple Interpreter: A Step-by-Step Guide https://devforum.roblox.com/t/building-a-simple-interpreter-a-step-by-step-guide/3310464
278. The Spectacular Interpreted Special (Ruby, Python and Lua) https://medium.com/@mario.arias.c/comparing-implementations-of-the-monkey-language-viii-the-spectacular-interpreted-special-ruby-2f9e4ed2e660
279. How to build your own language https://indico.cern.ch/event/769263/contributions/3406057/attachments/1834742/3017150/hands-on-demos.pdf
280. Creating a compiler or interpreter for the Contran language https://www.facebook.com/groups/selftaughtprogrammers/posts/478305822533157/
281. Tiny DSL implementation in Python https://stackoverflow.com/questions/31080796/tiny-dsl-implementation-in-python
282. I Made A LUA-BASED Programming Language https://www.youtube.com/watch?v=FFM-fKuWwHg
283. Unattended robot failing to ui elements when it was ... https://forum.uipath.com/t/unattended-robot-failing-to-ui-elements-when-it-was-installed-in-vm-showing-error-while-running-a-process-containing-action-center-when-published/622153
284. Interactive state doesn't reset after overlay is closed https://forum.figma.com/report-a-problem-6/interactive-state-doesn-t-reset-after-overlay-is-closed-31116

285. Blue Prism UI Automation Failing after Chrome / Edge Update ... https://community.blueprism.com/t5/Product-Forum/Blue-Prism-UI-Automation-Failing-after-Chrome-Edge-Update-to-140/td-p/122492/page/5
286. How can I change the default version of Python Used by ... https://stackoverflow.com/questions/50667214/how-can-i-change-the-default-version-of-python-used-by-atom
287. Automation mode restart = not restarting - Configuration https://community.home-assistant.io/t/automation-mode-restart-not-restarting/938943
288. How to Make Full Screen and Resizable Python Games with ... https://www.youtube.com/watch?v=GX_fsDz4j8A
289. PAGE - A Python GUI Generator https://page.sourceforge.net/
290. Library for (web based?) GUI that can show widgets, graphs ... https://discuss.python.org/t/library-for-web-based-gui-that-can-show-widgets-graphs-and-update-information/24537
291. ZCban/BetterCam: World's Fastest high-performance ... https://github.com/ZCban/BetterCam
292. Summary of Settings to Minimize PC Game Latency ... https://note.com/noted_pothos7691/n/n3016e5312a4f?hl=en
293. I'm personally too anxious to download any game with anti- ... https://www.reddit.com/r/pcmasterrace/comments/1pj39qy/im_personally_too_anxious_to_download_any_game/
294. [Help] DX11 Overlay – Best Way to Stay Undetected (ImGui ... https://www.unknowncheats.me/forum/anti-cheat-bypass/709175-dx11-overlay-stay-undetected-imgui-transparent-window.html
295. What are actual solutions to the anti cheat issue? https://www.reddit.com/r/linux_gaming/comments/1mg9v7g/what_are_actual_solutions_to_the_anti_cheat_issue/
296. Dear ImGui: Bloat-free Graphical User interface ... https://github.com/ocornut/imgui
297. Screen Scraping for our TFT Game AI with OCR | Teamfight ... https://www.youtube.com/watch?v=mw904WVfjJ4
298. SphinxNumberNine/valoscribe https://github.com/SphinxNumberNine/valoscribe
299. How similar are Lua & Python? https://www.reddit.com/r/lua/comments/1nwtw2j/how_similar_are_lua_python/
300. How to Choose Between Lua vs Python | Linode Docs https://www.akamai.com/cloud/guides/lua-vs-python/
301. Calling python and C++ scripts within a lua script https://stackoverflow.com/questions/57027448/calling-python-and-c-scripts-within-a-lua-script

302. bastibe/lunatic-python: A two-way bridge between ... https://github.com/bastibe/lunatic-python
303. 5 Python Libraries You Should Avoid If You Care About ... https://python.plainenglish.io/5-python-libraries-you-should-avoid-if-you-care-about-speed-some-libraries-look-fancy-on-github-d686f25aec55
304. Lua vs Python scripts https://manual.coppeliarobotics.com/en/luaPythonDifferences.htm
305. uhub/awesome-lua https://github.com/uhub/awesome-lua
306. A curated list of awesome Lua frameworks, libraries and ... https://github.com/forhappy/awesome-lua
307. Embedded Scripting Languages https://caiorss.github.io/C-Cpp-Notes/embedded_scripting_languages.html
308. 15 Best Lua Project Ideas for Beginners to Advanced Level https://bestassignmentgrade.com/blog/lua-project-ideas/
309. A curated list of amazingly awesome LÖVE libraries ... https://github.com/love2d-community/awesome-love2d
310. Always on top, transparent windows in PyQt6 : r/learnpython https://www.reddit.com/r/learnpython/comments/1t3b1ho/always_on_top_transparent_windows_in_pyqt6/
311. Transparent Windows With TKinter - Python Tkinter GUI ... https://www.youtube.com/watch?v=qDVxLMuNs7E
312. Set WindowStaysOnTopHint to work over fullscreen app https://forum.qt.io/topic/141043/set-windowstaysontophint-to-work-over-fullscreen-app
313. [Question] Transparent Click-through Windows in ... https://github.com/PySimpleGUI/PySimpleGUI/issues/4927
314. New Transparent Widget Hack With Tkinter - Python Tkinter ... https://www.youtube.com/watch?v=75jbNpc8vN4
315. PySide... Window Conundrum https://forum.logik.tv/t/pyside-window-conundrum/13073
316. how to: manually or automatically override any entity's state ... https://www.youtube.com/watch?v=Vt_pJ8dRBWA
317. Modern Python GUI Builder (2026) | Drag and drop GUI ... https://www.youtube.com/watch?v=wLJxseM1cvw
318. Everyone Says Python Can't Do GUIs — They're Wrong https://python.plainenglish.io/everyone-says-python-cant-do-guis-they-re-wrong-ce7c9ca630b0
319. Issues after Restart Server from UI #224 https://github.com/vladmandic/automatic/discussions/224

320. Automation state and "last triggered" not persisting across ... https://community.home-assistant.io/t/automation-state-and-last-triggered-not-persisting-across-reboots-suddenly/986480

321. Bot got stuck in the middle of transaction without any error ... https://forum.uipath.com/t/bot-got-stuck-in-the-middle-of-transaction-without-any-error-or-timeout-unexpected-freeze/4962534

322. Dynamic GUI creation using configuration files https://stackoverflow.com/questions/956368/dynamic-gui-creation-using-configuration-files

323. How to build a GUI for a Stable Diffusion texture generator ... https://www.youtube.com/watch?v=XstMgVMODY4

324. The Only Python GUI Library I'd Recommend to Beginners https://blog.stackademic.com/the-only-python-gui-library-id-recommend-to-beginners-11a462b657d2

325. DX12 + Optimizations for windowed games tanks ... https://www.reddit.com/r/wow/comments/1mmy4wy/dx12_optimizations_for_windowed_games_tanks/

326. tkinter was shockingly easy to write a small overlay GUI https://www.reddit.com/r/Python/comments/op1tz0/tkinter_was_shockingly_easy_to_write_a_small/

327. Getting started with Desktop Overlays with Python and Tkinter https://medium.com/@skash03/getting-started-with-desktop-overlays-with-python-and-tkinter-bfa92a23cf0

328. How to Create Transparent Windows using Python! https://www.youtube.com/watch?v=Hd2q1G8jO7s

329. Transparent window in Tkinter https://www.geeksforgeeks.org/python/transparent-window-in-tkinter/

330. How to build combined interfaces and dynamic UI for Python ... https://www.youtube.com/watch?v=GeiYsXWvX3o

331. I Tried Every Python GUI Framework So You Don't Have To https://python.plainenglish.io/i-tried-every-python-gui-framework-so-you-dont-have-to-1279a23fc947

332. How to make your Python GUI look awesome | by Sourav De https://medium.com/@SrvZ/how-to-make-your-python-gui-look-awesome-9372c42d7df4

333. 3 UI frameworks for writing user-friendly applications in ... https://www.redhat.com/en/blog/write-GUI-applications-python

334. How to build a cross-platform graphical user interface with ... https://www.youtube.com/watch?v=-HlWDVuZbYU

335. FrameKit: A Tool for Authoring Adaptive UIs Using Keyframes https://dl.acm.org/doi/fullHtml/10.1145/3640543.3645176

336. GuiProgramming - Python Wiki https://wiki.python.org/moin/GuiProgramming

337. #1 Setup: Building Modern GUIs using Python | Full Course ... https://www.youtube.com/watch?v=EIwu8B3fvSU

338. Python - Automate Programs With UIAutomation - Crazy Simple! https://www.youtube.com/watch?v=qXsRh2k_0zI

339. transitions-gui - A frontend for transitions state machines https://github.com/pytransitions/transitions-gui

340. Do you need GUI Automation? Do many situations demand ... https://www.reddit.com/r/Python/comments/wuq01n/do_you_need_gui_automation_do_many_situations/

341. Python libraries for infrastructure automation https://www.youtube.com/watch?v=2L2cVQWdv7Q

342. Python GUI automation library for simulating user ... https://softwarerecs.stackexchange.com/questions/42664/python-gui-automation-library-for-simulating-user-interaction-in-apps

343. These 4 Python UI Libraries Took My Automation Scripts to ... https://python.plainenglish.io/these-4-python-ui-libraries-took-my-automation-scripts-to-the-next-level-9de73e7c6323

344. Which Python extension for Windows GUI automation ... https://stackoverflow.com/questions/8752996/which-python-extension-for-windows-gui-automation-offers-the-most-flexibility-fo

345. Pi: The Minimal Agent Within OpenClaw https://lucumr.pocoo.org/2026/1/31/pi/

346. Is Lua similar to Python? Is it easier than Python? https://www.quora.com/Is-Lua-similar-to-Python-Is-it-easier-than-Python

347. monkt/paddleocr-onnx https://huggingface.co/monkt/paddleocr-onnx

348. LightOnOCR-2-1B: a lightweight high-performance end-to- ... https://huggingface.co/blog/lightonai/lightonocr-2

349. gmh5225/AI-FPS-b00m-h3adsh0t: Computer Vision Game ... https://github.com/gmh5225/AI-FPS-b00m-h3adsh0t

350. BOT-MMORPG-AI/USAGE.md at master https://github.com/ruslanmv/BOT-MMORPG-AI/blob/master/USAGE.md

351. OpenCV Object Detection in Games Python Tutorial #1 https://www.youtube.com/watch?v=KecMlLUuiE4

352. Cross-Platform UI Automation Framework for Games and Apps https://airtest.readthedocs.io/en/latest/README_MORE.html

353. onnx/models: A collection of pre-trained, state-of-the-art ... https://github.com/onnx/models

354. Clicking through a Translucent Image Window https://shallowsky.com/blog/programming/click-thru-translucent-update.html

355. Python Auto GUI (using PyQt) - Create GUIs with Just 1 Line of ... https://www.youtube.com/watch?v=U0Bso5jsC1g

356. Building Data Science GUI Apps with PySimpleGUI https://medium.com/data-science/building-data-science-gui-apps-with-pysimplegui-179db54a9a15

357. The Battle Between PYTHON GUI Frameworks | Episode #16 https://www.youtube.com/watch?v=FsSXKm2zwMY

358. Simeon Emanuilov's Post https://www.linkedin.com/posts/simeon-emanuilov_most-people-assume-ocr-for-document-processing-activity-7415783273483739136-Otgx

359. 20 computer vision projects with Python and OpenCV https://www.youtube.com/watch?v=jkevWUGOP4w

360. 2.1 Lua vs Python vs C++ programming (Coppeliasim ... https://www.youtube.com/watch?v=XODmc2AwAR4

361. Converting Python Code to LUA ... ? https://forums.civfanatics.com/threads/converting-python-code-to-lua.378560/

362. Create Python GUIs In 5 Minutes With This Package! https://www.youtube.com/watch?v=Szi8OgdWDWY

363. The Python GUI Library That Finally Made My Apps Look ... https://blog.stackademic.com/the-python-gui-library-that-finally-made-my-apps-look-professional-2609d25ca6ff

364. Create a directly-executable cross-platform GUI app using ... https://stackoverflow.com/questions/2933/create-a-directly-executable-cross-platform-gui-app-using-python

365. Is there a "good" way to build a GUI for Python? https://www.reddit.com/r/learnpython/comments/oms1px/is_there_a_good_way_to_build_a_gui_for_python/

366. Choosing a GUI Framework / Tool Kit / Libraries - Python ... https://www.youtube.com/watch?v=S7KBx3z6gJI